Estudio
Twiinking

PROMPTS
EFICIENTES

EL EQUIPO



Cristina Ramos
Gerente



Antonio Heredia
Diseño de interiores



Sergio Hernandez
Ilustrador

TABLA DE
CONTENIDO

- Introducción **01-04**
- Plan de acción **05**
- Cristina: Primer Prompt **06**
- Optimización del Prompt **07-09**
- Sergio: Primer Prompt **10**
- Optimización del Prompt **11-13**
- Antonio: Primer Prompt **14**
- Optimización del Prompt **15-17**
- Conclusión **18**

INTRODUCCIÓN
AL TRABAJO

En este trabajo se pretende demostrar como un prompt, aunque refiriendose a lo mismo puede variar enormemente el resultado de la respuesta que nos puede dar una IA.

Mostraremos 3 casos, desde el primer prompt inicial hasta el resultado final y la conclusión de como afecta la calidad de la instrucción dada a la IA.



PLAN DE ACCIÓN

Una vez aclarado la petición que se le hará a la IA hay que tener en cuenta algunos factores que mejorarán drásticamente el resultado.



Incluir contexto



Definir el tono



Restricciones



Especificar Salida



Ser explícito

PRIMER PROMPT
CRISTINA

Code Review

Crear un protocolo técnico de revisión de código (Code Review) para verificar la seguridad contra inyecciones SQL en un backend.

Herramienta utilizada: Google Gemini.

Prompt inicial

«Crea un protocolo de revisión de código para comprobar la seguridad contra inyecciones SQL en un backend...»

Genera un protocolo general con medidas para prevenir inyecciones SQL y ejemplos de código en Python. Sin embargo, no está adaptado a un público concreto ni a una tecnología específica.

EVOLUCIÓN DEL **PROMPT**

Segundo Prompt

«Actúa como un desarrollador especializado en seguridad informática. Crea un protocolo técnico de revisión de código dirigido a desarrolladores junior...»

Genera una guía más práctica y comprensible, con pasos para detectar vulnerabilidades y una checklist final. Sin embargo, sigue mezclando diferentes lenguajes de programación.

Prompt y respuesta

«Actúa como un desarrollador senior especializado en seguridad de aplicaciones web. Crea un protocolo técnico de Code Review para un backend desarrollado con PHP y MySQL...»

Genera un protocolo adaptado a PHP y MySQL, con ejemplos de código vulnerable y seguro, cinco apartados definidos y una checklist final de revisión.

PROMPT
FINAL





> Actúa como un desarrollador senior especializado en seguridad de aplicaciones web. Crea un protocolo técnico de Code Review dirigido a desarrolladores junior para verificar la seguridad contra inyecciones SQL en un backend desarrollado con PHP y MySQL.

> Organiza la respuesta en Markdown y divídela en cinco apartados:
>
> 1. Objetivo del protocolo.
> 2. Pasos principales que debe seguir el desarrollador durante la revisión.
> 3. Ejemplos sencillos en PHP de código vulnerable y su versión segura utilizando consultas preparadas.
> 4. Errores frecuentes que debe detectar el desarrollador.
> 5. Checklist final para determinar si el código supera la revisión.

> Utiliza un lenguaje técnico pero comprensible para desarrolladores junior. Explica brevemente el motivo de cada comprobación y limita la respuesta a los aspectos esenciales necesarios para realizar el Code Review, evitando información avanzada que no sea imprescindible.

EVOLUCIÓN DE LAS RESPUESTAS

Primera

1. Protocolo de Revisión Paso a Paso

- **Localizar la capa de datos:** Identifica todos los reposito
modelos encargados de ejecutar consultas SQL. PDF
- **Revisar la construcción de consultas:** Busca cadenas d
concatenación manual (`+` , `.` , `format()` , f-strings) pa
- **Validar parámetros:** Asegúrate de que cualquier dato d
exclusivamente a través de marcadores de posición (*pla*
librería/ORM. PDF
- **Controlar consultas dinámicas:** Verifica que los nombr
búsquedas avanzadas no provengan directamente del u
mediante una lista blanca (*allowlist*). PDF
- **Auditar procedimientos almacenados:** Comprueba qu
no ejecuten código SQL dinámico de forma interna mec

2. Aspectos Clave y Prevención

- **Uso de Sentencias Preparadas:** Es la defensa principal,
los datos de entrada para evitar que el usuario altere la e
- **Uso Adecuado de ORM:** Prioriza los métodos nativos de
en lugar de consultas nativas (*raw queries*) que puedan
PDF
- **Principio de Menor Privilegio:** Configura la conexión de
con los permisos mínimos indispensables, evitando cuer
administrador o `root` . PDF +1

Segunda

1. Protocolo de Revisión Paso a Paso

- **Ubicar puntos de entrada:** Identifica dónde entran datos externos (URLs, cuerpo de peticiones HTTP, cabeceras, cookies, o parámetros de ordenación). PDF
- **Seguir el recorrido del dato (*Taint Analysis*):** Rastrea el flujo de la variable desde el controlador o ruta hasta la capa de persistencia. PDF
- **Analizar la construcción de la consulta:** Inspecciona si la sentencia SQL utiliza concatenación/formateo (bandera roja) o si emplea marcadores de posición (*placeholders*) y métodos seguros del ORM. PDF

2. Detección de Vulnerabilidades Comunes

- **Concatenación o formateo de cadenas:** Detecta el uso de símbolos de suma (`+`), interpolación (`f"..."`) o funciones como `format()` para armar consultas, ya que permiten inyecciones directas. PDF +1
- **Consultas nativas (*Raw Queries*) en ORMs:** Revisa si se usan funciones de SQL directo dentro de ORMs (como Sequelize, TypeORM o SQLAlchemy) pasando texto sin desinfectar. PDF
- **Ordenación o nombres de tablas dinámicos:** Comprueba si se insertan variables del usuario directamente en cláusulas como `ORDER BY` , donde los marcadores de posición estándar no funcionan. PDF

3. Corrección y Soluciones

- **Consultas parametrizadas:** Separa la estructura de la consulta de los datos de entrada usando marcadores como `%s` o `$1` para que el motor de la base de datos los trate estrictamente como datos. PDF
- **Funciones nativas del ORM:** Prioriza el uso de la API propia del ORM en lugar de escribir SQL manual. PDF
- **Listas blancas (*Allowlists*):** Valida los valores dinámicos (como columnas de ordenación) contra una lista estricta predefinida antes de incluirlos en la consulta. PDF +1

4. *Checklist* Rápida para el Revisor

- [ ] No existen concatenaciones de texto en ninguna instrucción SQL. PDF
- [ ] Todas las variables externas se pasan como parámetros independientes. PDF
- [ ] Las consultas nativas en ORM fueron sustituidas por métodos de la API o parametrizadas. PDF
- [ ] Los nombres de columnas o tablas dinámicas se validan obligatoriamente con una lista blanca. PDF

Tercera

1. Pasos Principales durante la Revisión

- **Rastrear la entrada de datos:** Localiza variables externas provenientes de `$_GET` , `$_POST` , `$_COOKIE` , `$_REQUEST` o cabeceras HTTP. PDF
- **Seguir el flujo de datos:** Verifica cómo viajan esos datos desde el controlador hasta la capa donde se construyen las consultas SQL. PDF
- **Inspeccionar la sentencia SQL:** Analiza la línea de la consulta para comprobar si los datos del usuario se concatenan o se pasan como parámetros aislados. PDF
- **Verificar la API de base de datos:** Asegúrate de que se utilicen librerías modernas como **PDO** o **MySQLi** configuradas con consultas preparadas (*Prepared Statements*). PDF

2. Ejemplos Clave: Vulnerable vs. Seguro

- **Consultas de selección (`SELECT`):**
  - *Inseguro:* Interpolación directa de variables en la cadena SQL (permite inyectar comandos como `'OR '1'='1`). PDF +1
  - *Seguro:* Uso de marcadores de posición (como `:email` en PDO) vinculando el valor de forma separada. PDF
- **Inserción de datos (`INSERT`):**
  - *Inseguro:* Concatenación directa de múltiples campos de formularios con `mysqli` . PDF
  - *Seguro:* Uso de marcadores de posición en forma de interrogación (`?`) y asignación mediante `bind_param` . PDF

3. Errores Frecuentes a Detectar

- **Confiar en escapes manuales:** Usar `addslashes()` o `mysqli_real_escape_string()` como única defensa es propenso a fallos y no sustituye a las consultas preparadas. PDF
- **Omitir marcadores en campos numéricos:** Pensar que `WHERE id = $id` es seguro por ser un número; si no se fuerza un *cast* a entero (`(int)$id`), sigue siendo vulnerable. PDF
- **Concatenar en cláusulas dinámicas:** Las consultas preparadas no soportan parámetros en nombres de tablas o en cláusulas como `ORDER BY` o `GROUP BY` , por lo que se debe recurrir obligatoriamente a una lista blanca (*allowlist*). PDF
- **Emulaciones inseguras en PDO:** Mantener activado `PDO::ATTR_EMULATE_PREPARES` de forma incorrecta puede abrir brechas según la configuración del conjunto de caracteres. PDF

4. Checklist Final de Aprobación

- [ ] **Cero concatenaciones:** Ninguna variable dinámica se concatena ni se interpola dentro

PRIMER PROMPT
SERGIO

Cuestionario/Checklist inicial para app móvil

Diseñar un cuestionario/Checklist para recopilar requerimientos funcionales del cliente en la fase inicial de desarrollo de una app móvil.

Primer prompt

Herramienta utilizada: **Gemini.**

- Diséñame un cuestionario/Checklist para recopilar requerimientos funcionales del cliente en la fase inicial de desarrollo de una app móvil.

ANÁLISIS DE LA **PRIMERA RESPUESTA**

Aspectos positivos

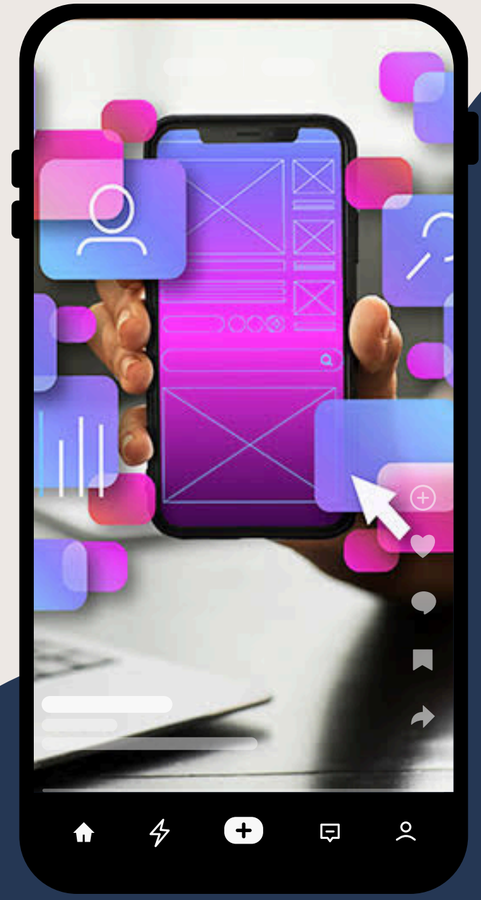
- Está organizada en ocho apartados y es fácil de seguir.
- Incluye muchas funcionalidades habituales de una app móvil.
- Utiliza listas y casillas para facilitar la revisión.

Aspectos mejorables

- Utiliza demasiados términos técnicos (CRUD, API REST, SSO...).
- Incluye funciones que no todas las aplicaciones necesitan.
- Faltan preguntas más directas para conocer las necesidades del cliente.
- No distingue entre funciones imprescindibles y opcionales.
- Mezcla requisitos funcionales con aspectos técnicos y comerciales.

PROMPT **FINAL**

Actúa como un analista de requisitos de software. Diseña un cuestionario/checklist para que un equipo de desarrolladores y analistas pueda recopilar los requisitos funcionales de un cliente antes de comenzar el desarrollo de una aplicación móvil. El cuestionario debe ayudar a conocer los usuarios de la aplicación, sus funciones principales, registro e inicio de sesión, gestión de datos, notificaciones, pagos y otras necesidades del cliente. Presenta la información en formato Markdown, organizada por apartados y utilizando casillas de verificación (□) para que pueda utilizarse como checklist. Utiliza un tono profesional y pedagógico, con preguntas sencillas que el cliente pueda entender. Evita la jerga técnica y no incluyas código ni aspectos relacionados con la implementación técnica.



ANÁLISIS DEL PROMPT FINAL

- Mejor organización
- Preguntas más concretas
- Mayor atención al usuario
- Recopilación de requisitos más completa
- Formato práctico
- Lenguaje más comprensible

EVOLUCIÓN DE LAS RESPUESTAS

Primera

Checklist de Recopilación de Requerimientos Funcionales (App Móvil)

1. Visión del Producto y Objetivos

- Propósito principal: ¿Qué problema clave resuelve la aplicación y cuál es su propuesta de valor única?
- Métricas de éxito (KPIs): ¿Cómo se medirá el éxito de la app? (ej. usuarios activos diarios, conversiones, retención, tiempo de sesión).
- Público objetivo: ¿Quiénes son los usuarios finales? (Edad, ubicación, contexto de uso, nivel de alfabetización digital).

2. Gestión de Usuarios y Autenticación

- Registro e Inicio de Sesión:
  - ☐ Correo electrónico y contraseña
  - ☐ SSO / Social Login (Google, Apple, Facebook)
  - ☐ Verificación en dos pasos (2FA vía SMS, email o app de autenticación)
  - ☐ Autenticación biométrica (Face ID / Huella dactilar)
- Gestión de Perfiles:
  - ☐ Edición de datos personales, foto de perfil y preferencias de la cuenta
  - ☐ Roles y permisos diferenciados (ej. Cliente, Administrador, Repartidor, Vendedor)
  - ☐ Opción de eliminación de cuenta (obligatorio para Apple Store y Google Play)

3. Funcionalidades Principales (Core Features)

- Navegación y Estructura:
  - Pantalla de inicio / Dashboard personalizado al iniciar sesión
  - Estructura de navegación principal (Barra inferior, menú lateral o pestañas)
- Búsqueda y Exploración:
  - ☐ Búsqueda por texto con auto-completado
  - ☐ Filtros avanzados y ordenamiento (por precio, fecha, relevancia, cercanía)
- Gestión de Contenido y Transacciones:
  - ☐ Creación, edición y eliminación de registros (POST/CRUD de usuarios)
  - ☐ Carrito de compras, sistema de reservas o solicitudes de servicio
  - ☐ Guardado de favoritos, marcadores o historial de actividad

4. Notificaciones y Comunicación

- Notificaciones Push:
  - ☐ Transaccionales (confirmaciones de compra, cambios de estado de pedido)
  - ☐ Promocionales y re-engagement (ofertas, recordatorios)
  - ☐ Basadas en geocercas / ubicación (Geofencing)
- Mensajería e Interacción:
  - ☐ Chat en tiempo real (entre usuarios o con atención al cliente)
  - ☐ Envío de archivos, imágenes o notas de voz dentro de la app

Segunda

Cuestionario de Recopilación de Requisitos Funcionales para Aplicaciones Móviles

Este cuestionario sirve como guía de trabajo en la fase inicial de toma de requerimientos. Está diseñado para presentarse directamente al cliente, con un lenguaje claro y libre de tecnicismos, orientado a definir qué debe hacer la aplicación.

1. Visión General y Perfil de Usuarios

☐ Objetivo Principal: ¿Cuál es la función principal de la aplicación y qué problema específico resuelve?

☐ Público Objetivo: ¿Quiénes utilizarán la aplicación? (Ejemplo: clientes finales, empleados de la empresa, proveedores, distribuidores).

☐ Tipos de Usuario: ¿Existirán distintos roles dentro de la aplicación con diferentes permisos?

☐ Ejemplos de roles: Usuario comprador, vendedor, repartidor, administrador.

☐ Idiomas: ¿La aplicación se publicará en un solo idioma o debe contar con soporte multidioma?

2. Registro, Inicio de Sesión y Perfiles de Usuario

☐ Acceso Inicial: ¿Es obligatorio que el usuario se registre para usar la app, o puede explorar el contenido de manera anónima antes de registrarse?

☐ Métodos de Registro e Inicio de Sesión: ¿Cómo prefiere que los usuarios creen su cuenta?

☐ Correo electrónico y contraseña

☐ Número de teléfono (con código de confirmación vía SMS)

☐ Cuentas existentes (Google, Apple, Facebook)

☐ Acceso biométrico (Huella dactilar / Reconocimiento facial)

☐ Datos del Perfil: ¿Qué información personal recopilará el usuario en su perfil? (Nombre, foto, dirección de entrega, teléfono, datos de facturación).

☐ Recuperación de Cuenta: ¿Se requiere un flujo simple de "Olvidé mi contraseña" vía correo o SMS?

☐ Gestión de Cuenta: ¿El usuario tendrá la opción de modificar sus datos o eliminar su cuenta de forma autónoma?

3. Funcionalidades Principales y Flujo de Trabajo

☐ Pantalla Inicial: ¿Qué información o sección debe ver el usuario inmediatamente al abrir la aplicación?

☐ Acciones Clave: ¿Cuáles son las tres tareas más importantes que el usuario debe poder completar en la app?

☐ Búsqueda y Exploración: ¿Cómo buscarán la información los usuarios dentro de la app?

☐ Búsqueda por texto directo

☐ Filtros por categorías, precio, fecha o cercanía

☐ Ordenamiento de resultados (por popularidad, menor precio, más reciente)

☐ Interacciones Frecuentes: ¿La aplicación requiere alguna de las siguientes funciones?

☐ Guardar elementos en "Favoritos" o listas de deseos

☐ Agendar citas o realizar reservas en un calendario

☐ Calificar o dejar reseñas sobre productos, servicios o usuarios

☐ Compartir contenidos en redes sociales o aplicaciones externas

4. Gestión de Datos y Contenido

☐ Origen del Contenido: ¿Quién creará y actualizará el contenido que se muestra en la aplicación? (El equipo interno, los mismos usuarios, o se actualizará automáticamente desde otro sistema).

☐ Carga de Archivos: ¿Los usuarios necesitan adjuntar información desde su teléfono?

☐ Fotografías o imágenes de la galería

☐ Documentos (PDF, recibos, archivos de texto)

☐ Notas de voz o grabaciones de audio

☐ Historial de Actividad: ¿El usuario necesita un panel donde pueda consultar sus movimientos pasados? (Ejemplo: historial de compras, servicios solicitados, mensajes enviados).

PRIMER PROMPT
ANTONIO

Guía de Github

Redactar un documento técnico en markdown que explique las mejores prácticas de nomenclatura commits e issues en GitHub para un equipo de desarrolladores junior.

Prompt y respuesta

Hazme una guía de GitHub

Genera una guía inicial básica de como funciona la página de manera fundamental, además lo genera como una respuesta en vez de archivo.

EVOLUCIÓN DEL **PROMPT**

Segundo

Hazme una guía sobre la nomenclatura de commits e issues en GitHub.

Ahora si que empieza a aportar ejemplos y recomendaciones a la hora de escribir issues y commits incluso algunas plantillas para tener la estructura a la vista

Final

Actua como un tech Lead...

Genera un archivo muy completo de manera muy bien explicada y sin dejar nada sobre todo el tema que se le ha pedido

PROMPT
FINAL



Actúa como un desarrollador web con experiencia en Git y GitHub. Redacta un documento técnico en español, en formato Markdown, para un equipo de desarrolladores junior que trabaja de forma colaborativa en proyectos de software.
Explica buenas prácticas para nombrar y redactar commits e issues de GitHub. Organiza el documento con títulos y subtítulos e incluye:
Una explicación breve de qué son los commits y las issues, y para qué sirven.

Una convención sencilla para los mensajes de commit, con una plantilla y ejemplos correctos e incorrectos. Usa como referencia tipo: descripción breve y explica tipos habituales como feat, fix, docs, test y refactor.
Una plantilla recomendada para crear issues, con campos como título, contexto, descripción del problema o necesidad, pasos para reproducirlo cuando corresponda, resultado esperado y criterios de aceptación.
Ejemplos de títulos y descripciones bien redactados para distintos tipos de issue, como error, nueva funcionalidad y documentación.

Recomendaciones prácticas de claridad, brevedad, idioma coherente y relación entre commits, issues y pull requests.
Mantén un tono pedagógico y directo. Define cualquier término técnico la primera vez que aparezca. No presentes estas convenciones como reglas oficiales obligatorias de GitHub; aclara que son acuerdos recomendados que el equipo puede adaptar. Evita explicaciones extensas y limita el documento a unas 800 palabras.

EVOLUCIÓN DE LAS RESPUESTAS

Primera

uía básica de GitHub

itHub es una plataforma para guardar y gestionar proyectos de software en línea. arias personas colaboren en el mismo proyecto y mantengan un historial de los ca ealizados.

onceptos básicos

- **Repositorio:** espacio donde se almacenan los archivos de un proyecto.
- **Commit:** registro de uno o varios cambios realizados en el proyecto. Cada com un mensaje que resume esos cambios.
- **Rama (branch):** línea de trabajo independiente. Permite probar cambios sin m directamente la versión principal del proyecto.
- **Pull request:** propuesta para revisar e incorporar cambios de una rama a otra.
- **Issue:** tarea, error o propuesta relacionada con el proyecto. Sirve para organiz conversar sobre él.

lujo básico de trabajo

1. Crea un repositorio o únete a uno existente.
2. Descarga el proyecto en tu equipo o edítalo desde GitHub.
3. Crea una rama para trabajar en una tarea.
4. Realiza cambios y guárdalos en commits.
5. Abre una pull request para que otras personas revisen los cambios.
6. Cuando se aprueben, incorpora los cambios al proyecto.

onsejos

- Escribe mensajes de commit que expliquen brevemente los cambios.
- Usa ramas para trabajar en nuevas funciones o corregir errores.
- Revisa los cambios antes de incorporarlos al proyecto.
- Utiliza las issues para registrar tareas, errores y sugerencias.
- Mantén el repositorio organizado y añade un archivo README que explique

itHub ayuda a colaborar, organizar tareas y c ervar el historial de un proyecto.

Segunda

Mensajes de commit

Un commit registra cambios realizados en el proyecto. Su mens breve y clara qué se ha modificado.

Recomendaciones

- Empieza con un verbo en imperativo, como «Añade», «Co
- Describe un cambio concreto; evita mensajes vagos como
- Mantén el título breve y añade más detalles en la descripci
- Sigue el mismo idioma y estilo en todo el repositorio.
- Evita incluir varios cambios no relacionados en un mismo

Ejemplos

Poco claro: `cambios varios`
Más claro: `Corrige la validación del formulario de regist`

Poco claro: `fix`
Más claro: `Actualiza el texto de ayuda de la página de in`

Si el equipo prefiere una convención basada en tipos, puede util `filtro de búsqueda` o `fix: corregir el cálculo del total`.

Títulos y descripciones de issues

Una issue sirve para describir una tarea, un error o una propues entender rápidamente de qué trata.

Recomendaciones

- Escribe un título específico, no demasiad neral.

Tercera

ipos recomendados

Tipo	Cuándo usarlo	Ejemplo
`feat`	Se añade una nueva funcionalidad	`feat: añadir filtro por categoría`
`fix`	Se corrige un error	`fix: corregir cálculo de del carrito`
`docs`	Solo cambia documentación	`docs: actualizar instala README`
`test`	Se crean o modifican pruebas	`test: añadir casos para registro de usuario`
`refactor`	Se reorganiza código sin cambiar su comportamiento	`refactor: separar valida del formulario`

Buenas prácticas

- Describe una única intención principal por commit.
- Utiliza una frase corta, clara y específica.
- Mantén el mismo idioma en todos los mensajes del repositorio.
- Escribe en minúsculas después de los dos puntos, salvo nombres propios.
- Evita información irrelevante, como el nombre de quien ha realizado el cambio.
- Si el cambio pertenece a una tarea concreta, indica su referencia cuando el equip acuerde: `fix: corregir validación de correo (#24)`.

Ejemplos correctos e incorrectos

Poco recomendable	Recomendado	Motivo
`cambios`	`feat: añadir página de perfil`	El segundo explica ha añadido.
`fix`	`fix: evitar envío sin correo electrónico`	El segundo identif error corregido.

CONCLUSIÓN

Al comparar las respuestas que nos ha ido dando la IA, hemos podido comprobar que mejorar las instrucciones permite obtener un resultado más adaptado al objetivo que se busca y queda claro que la IA, de manera predeterminada, en caso de no saber algo decide adivinar de manera bastante confiada antes que preguntar.

Las primeras versiones eran breves y generales. Aunque en algunas acertó y en ciertas partes estaba bastante completa, la respuesta incluís muchas elementos que no se requerían o eran prescindibles o de un nivel muy superior al necesario, debido a la falta de contexto en el prompt. Con las versiones consecutivas se iba acercando más y más al resultado esperado, aumentaba la cantidad de elementos que se buscaban, mejoraba el formato y el tipo de lenguaje. Como resultado obtuvimos respuestas muy organizadas y muy concretas, perfectas para cubrir las necesidades que buscabamos.

La mejora no consiste simplemente en obtener una respuesta más larga, sino en conseguir que sea más clara y útil para la tarea que queremos realizar. Sin embargo, la segunda respuesta también tiene aspectos que se podrían perfeccionar, por lo que siempre es importante revisar el contenido generado por la IA.



Estudio
Twiinking

MUCHAS GRACIAS

Por escucharnos en esta, nuestra presentación acerca del refinamiento y el correcto uso de los prompts que se le redactan a la IA con el propósito de mejorar vuestra experiencia con dichas herramientas

01010011 01001111 01010011

www.unsitiogenial.es

Página de Recursos



CONCLUSIÓN

Al comparar las dos respuestas, hemos comprobado que mejorar las instrucciones permite obtener un resultado más adaptado al objetivo que se busca.

El primer prompt era breve y general. Aunque generó un checklist bastante completo, la respuesta incluía muchos términos técnicos y enumeraba características sin centrarse tanto en las preguntas que necesita responder un cliente.
El segundo prompt definía el rol de la IA, el contexto de uso, los apartados que debía incluir, el formato y el tipo de lenguaje. Como resultado, obtuvimos un cuestionario mejor organizado y con preguntas más concretas para conocer las necesidades, las prioridades y las funcionalidades de la aplicación.

La mejora no consiste simplemente en obtener una respuesta más larga, sino en conseguir que sea más clara y útil para la tarea que queremos realizar. Sin embargo, la segunda respuesta también tiene aspectos que se podrían perfeccionar, por lo que siempre es importante revisar el contenido generado por la IA.

